PHASE 10 OF 18

Nested Classes, Enums & Generics

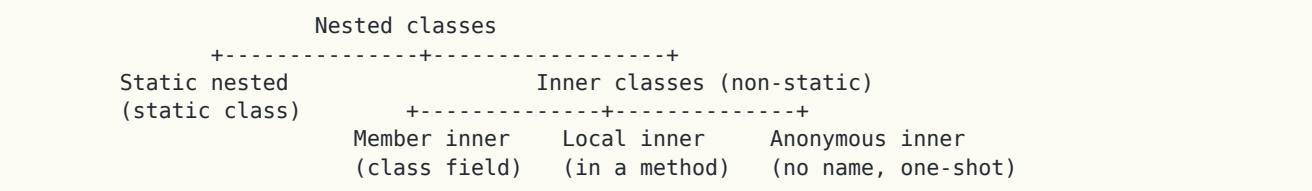
Static nested, member inner, local and anonymous classes, anonymous classes vs lambdas, enums with fields and behaviour, EnumMap/EnumSet, generic classes and methods, bounds, wildcards and PECS, and type erasure

IN THIS PHASE

10.1 Nested Classes: The Four Kinds
10.2 Member Inner Class
10.3 Static Nested Class
10.4 Local Inner Class
10.5 Anonymous Inner Class
10.6 Enums
10.7 Generics
10.8 Type Erasure
10.9 Predict the Output
10.10 Interview Questions
10.11 Quick Revision

10.1 Nested Classes: The Four Kinds

A class declared inside another class is a **nested class**. Nested classes **group** helper types with the only class that uses them, **increase encapsulation** (they can be private and can access the outer class's private members), and keep code **readable**.



Kind	Declared	Needs an outer object?	Can access outer instance members?	Created as	Compiled to
Static nested	`static class` inside a class	No	No (only static ones, or via a reference)	`new Outer.Nested()`	`Outer$Nested.class`
Member inner	Non-static class inside a class	**Yes**	**Yes**, including private	`outer.new Inner()`	`Outer$Inner.class`
Local inner	Inside a method or block	Yes, if in an instance method	Yes, plus effectively-final locals	`new Local()` within that block	`Outer$1Local.class`
Anonymous inner	`new Type() { ... }` expression	Yes, if in an instance context	Yes, plus effectively-final locals	At declaration only	`Outer$1.class`

10.2 Member Inner Class

```
class Outer {
    private int x = 10;

    class Inner {                                   // tied to an Outer instance
        private int x = 20;
        void display() {
            int x = 30;
            System.out.println(x);                  // 30 (local)
            System.out.println(this.x);             // 20 (Inner's field)
            System.out.println(Outer.this.x);       // 10 (enclosing instance's field)
        }
    }

    void useInner() {
        new Inner().display();                      // inside Outer: no prefix needed
    }
}

Outer outer = new Outer();
Outer.Inner inner = outer.new Inner();              // outside: outer.new Inner()
inner.display();
```

- An inner class can access **all** members of the outer class, including private ones (and vice versa).
- Every inner object holds a hidden reference to its **enclosing instance** (`Outer.this`).
- Since Java 16, inner classes may also declare `static` members.

> **PITFALL**
> Because every inner-class object keeps its outer object alive, a long-lived inner object (stored in a cache, or passed to another thread) can cause **memory leaks**. If the nested class doesn't need the outer instance, make it **static**.

Real-world use: `ArrayList`'s iterator (`ArrayList.Itr`) is a private inner class that reads the list's internal array directly.

10.3 Static Nested Class

A static nested class behaves like a top-level class that is **namespaced** inside another class. It has no link to an outer instance.

```
class Outer {
    static int count = 10;
    int y = 20;

    static class Nested {
        void display() {
            System.out.println(count);     // OK: static member of Outer
            // System.out.println(y);      // COMPILE ERROR: needs an Outer instance
        }
    }
}
Outer.Nested n = new Outer.Nested();       // no Outer object needed
```

Classic use: the Builder pattern

```
public final class Customer {
    private final String name;          // required
    private final String email;         // optional
    private final String city;          // optional

    private Customer(Builder b) {
        this.name = b.name; this.email = b.email; this.city = b.city;
    }

    public static class Builder {                        // static nested
        private final String name;
        private String email, city;
        public Builder(String name) { this.name = name; }
        public Builder email(String e) { this.email = e; return this; }
        public Builder city(String c)  { this.city = c;  return this; }
        public Customer build()        { return new Customer(this); }
    }
}

Customer c = new Customer.Builder("Rohit").email("r@x.com").city("Pune").build();
```

Other JDK examples: `Map.Entry`, `AbstractMap.SimpleEntry`, `Character.UnicodeBlock`, and the nodes of `HashMap` (`HashMap.Node`).

10.4 Local Inner Class

A local class is declared **inside a method or block** and is visible only there. It can use the method's **effectively final** local variables.

```
class Validator {
    private final int minLength = 8;

    boolean validate(String password) {
        String special = "@#$%";                         // effectively final

        class Rule {                                     // local class
            boolean check() {
                return password.length() >= minLength                    // outer field
                    && password.chars().anyMatch(ch -> special.indexOf(ch) >= 0);
            }
        }
        // special = "!";   // would make it NOT effectively final -> compile error above
        return new Rule().check();
    }
}
```

> **MODERN JAVA**
> Java 16+ also allows **local records, enums and interfaces** inside methods. They are handy for short-lived data shapes in a single method.

10.5 Anonymous Inner Class

An anonymous class is a **nameless** class declared and instantiated in one expression. It either **implements an interface** or **extends a class** (concrete or abstract). It is used once, on the spot.

```
interface Greeting { void greet(); }

Greeting g = new Greeting() {                    // implements an interface
    @Override public void greet() { System.out.println("Hello, Anonymous!"); }
};
g.greet();

Thread t = new Thread() {                        // extends a class
    @Override public void run() { System.out.println("worker running"); }
};
t.start();

List<String> names = new ArrayList<>(List.of("Ravi", "Amit", "Zoya"));
names.sort(new Comparator<String>() {            // classic pre-Java 8 Comparator
    @Override public int compare(String a, String b) { return b.compareTo(a); }
});
System.out.println(names);                       // [Zoya, Ravi, Amit]
```

Anonymous class vs lambda

```
Runnable r1 = new Runnable() {                   // anonymous class
    @Override public void run() { System.out.println("Task"); }
};
Runnable r2 = () -> System.out.println("Task");  // lambda (Java 8): same behaviour
names.sort((a, b) -> b.compareTo(a));            // lambda comparator
names.sort(Comparator.reverseOrder());           // or a ready-made comparator
```

	Anonymous class	Lambda
Works with	Interfaces with **any** number of methods, abstract or concrete classes	**Functional interfaces only** (one abstract method)
Meaning of `this`	The anonymous object itself	The **enclosing** instance
Can have fields and state	Yes	No
Generated `.class` file	Yes (`Outer$1.class`)	No (`invokedynamic` at runtime)
Verbosity	High	Minimal
Captured locals	Must be effectively final	Must be effectively final

> **TIP**
> Use a lambda (or method reference) whenever the target is a functional interface. Use an anonymous class when you need state, several methods, or to extend a class.

Which nested type should I use?

Need	Use
A helper that doesn't need the outer object (Builder, Node, Entry, DTO)	**Static nested** (the default choice)
A helper that must read or write the outer object's state (an iterator over its data)	Member inner
A one-method-scoped helper with a name	Local class (or local record)
A one-off implementation with state or several methods	Anonymous class
A one-off implementation of a functional interface	**Lambda**

10.6 Enums

An **enum** is a special type representing a **fixed set of named constants**: days of the week, account types, transaction statuses, card networks.

```
enum TxnStatus { PENDING, APPROVED, REJECTED }

TxnStatus s = TxnStatus.APPROVED;
System.out.println(s);                              // APPROVED

switch (s) {                                        // works in switch (no prefix in labels)
    case PENDING  -> System.out.println("Request pending");
    case APPROVED -> System.out.println("Request approved");
    case REJECTED -> System.out.println("Request rejected");
}
```

Why enums instead of int or String constants?

```
// Old style: no type safety
static final int PENDING = 0, APPROVED = 1, REJECTED = 2;
void update(int status) { }          // update(42) compiles: invalid value accepted!

// Enum: the compiler only accepts real constants
void update(TxnStatus status) { }    // update(42) and update("APPROVED") won't compile
```

Benefits: type safety, a closed set of values, readable code, compatibility with `switch` (including exhaustive switch expressions), and the ability to carry **fields and behaviour**.

Facts about enums

1. Every enum implicitly **extends** `java.lang.Enum`, so it **cannot extend another class**, but it **can implement interfaces**.
2. Each constant is implicitly `public static final`, and exactly **one instance** exists per constant.
3. Constructors are implicitly **private**: you can't call `new TxnStatus()`.
4. Enums are implicitly **final**, unless a constant has a body.
5. You can compare constants safely with `==` (singletons), and `==` is null-safe.
6. Enums are `Comparable` (by declaration order) and `Serializable`.

Built-in methods

Method	Example	Result
`values()` (static)	`TxnStatus.values()`	`[PENDING, APPROVED, REJECTED]` (a new array each call)
`valueOf(String)` (static)	`TxnStatus.valueOf("APPROVED")`	`APPROVED`. Case-sensitive; unknown names throw `IllegalArgumentException`.
`name()`	`APPROVED.name()`	`"APPROVED"` (exact declared name; final)
`toString()`	`APPROVED.toString()`	`"APPROVED"` by default (overridable)
`ordinal()`	`APPROVED.ordinal()`	`1` (zero-based position)
`compareTo(e)`	`PENDING.compareTo(REJECTED)`	negative (declaration order)
`getDeclaringClass()`	`APPROVED.getDeclaringClass()`	`TxnStatus.class`

Enums with fields, constructors and methods

```
public enum TxnType {
    UPI("U01", "Unified Payments Interface", 0.00),
    NEFT("N01", "National Electronic Funds Transfer", 2.50),
    RTGS("R01", "Real Time Gross Settlement", 25.00),
    IMPS("I01", "Immediate Payment Service", 5.00);    // semicolon before members

    private final String code;
    private final String description;
    private final double fee;

    TxnType(String code, String description, double fee) {   // implicitly private
        this.code = code; this.description = description; this.fee = fee;
    }

    public String getCode()        { return code; }
    public String getDescription() { return description; }
    public double getFee()         { return fee; }

    private static final Map<String, TxnType> BY_CODE = new HashMap<>();
    static {
        for (TxnType t : values()) BY_CODE.put(t.code, t);   // fast reverse lookup
    }
    public static TxnType fromCode(String code) {
        TxnType t = BY_CODE.get(code);
        if (t == null) throw new IllegalArgumentException("Unknown code: " + code);
        return t;
    }
}

System.out.println(TxnType.NEFT.getFee());           // 2.5
System.out.println(TxnType.fromCode("R01"));         // RTGS
```

Constant-specific behaviour

```
enum Operation {
    PLUS("+")   { int apply(int a, int b) { return a + b; } },
    MINUS("-")  { int apply(int a, int b) { return a - b; } },
    TIMES("*")  { int apply(int a, int b) { return a * b; } };

    private final String symbol;
    Operation(String symbol) { this.symbol = symbol; }
    abstract int apply(int a, int b);         // each constant must implement it
    @Override public String toString() { return symbol; }
}

for (Operation op : Operation.values())
    System.out.println("6 " + op + " 3 = " + op.apply(6, 3));
// 6 + 3 = 9    6 - 3 = 3    6 * 3 = 18
```

Enum implementing an interface, and the enum singleton

```
interface FeePolicy { double feeFor(double amount); }

enum CardType implements FeePolicy {
    DEBIT  { public double feeFor(double amt) { return 0; } },
    CREDIT { public double feeFor(double amt) { return amt * 0.018; } }
}

public enum AppConfig {          // simplest thread-safe, serialization-safe singleton
    INSTANCE;
    private final Properties props = new Properties();
    public String get(String key) { return props.getProperty(key); }
}
AppConfig.INSTANCE.get("db.url");
```

EnumSet and EnumMap

Specialised, very fast collections (backed by bit vectors and arrays) for enum keys:

```
enum Day { MON, TUE, WED, THU, FRI, SAT, SUN }

EnumSet<Day> weekend  = EnumSet.of(Day.SAT, Day.SUN);
EnumSet<Day> weekdays = EnumSet.complementOf(weekend);       // MON..FRI
EnumSet<Day> midweek  = EnumSet.range(Day.TUE, Day.THU);

EnumMap<TxnStatus, Long> counts = new EnumMap<>(TxnStatus.class);
counts.merge(TxnStatus.APPROVED, 1L, Long::sum);
System.out.println(counts);                                  // {APPROVED=1}
```

PITFALL

Never **persist** `ordinal()` values (in a database, a file or an API). Inserting or reordering constants silently changes their meaning. Store `name()` or an explicit code field. In JPA, use `@Enumerated(EnumType.STRING)` instead of the default `ORDINAL`.

10.7 Generics

Generics (Java 5) let classes, interfaces and methods be **parameterised by type**. You get **compile-time type safety**, **no casts**, and **reusable** code.

```
// Without generics (raw types): anything goes, errors appear at RUNTIME
List list = new ArrayList();
list.add(10);
list.add("Hello");
String v = (String) list.get(0);      // ClassCastException at runtime

// With generics: errors caught at COMPILE time, no casts
List<String> names = new ArrayList<>();   // <> = diamond operator (Java 7), type inferred
names.add("Java");
// names.add(100);                    // COMPILE ERROR
String first = names.get(0);          // no cast needed
```

Term	Meaning	Example
Type parameter	Placeholder declared on a class or method	`T` in `class Box<T>`
Type argument	The actual type supplied	`String` in `Box<String>`
Parameterized type	Generic type with arguments	`List<String>`
Raw type	Generic type used without arguments (legacy; avoid)	`List`
Diamond	Let the compiler infer the arguments	`new HashMap<>()`

Naming conventions: `T` (type), `E` (element), `K`/`V` (key/value), `N` (number), `R` (result), `S`/`U` (second and third types).

Generic classes and interfaces

```
public class Box<T> {
    private T value;
    public void set(T value) { this.value = value; }
    public T get() { return value; }
}

Box<Integer> intBox = new Box<>();
intBox.set(10);
int n = intBox.get();                // no cast (auto-unboxing)

Box<String> strBox = new Box<>();
strBox.set("Java");

public class Pair<K, V> {            // several type parameters
    private final K key; private final V value;
    public Pair(K key, V value) { this.key = key; this.value = value; }
    public K getKey() { return key; }
    public V getValue() { return value; }
}
Pair<String, Integer> p = new Pair<>("age", 35);

public interface Repository<T, ID> {  // generic interface (like Spring Data's CrudRepository)
    T findById(ID id);
    List<T> findAll();
    T save(T entity);
}
class AccountRepo implements Repository<Account, Long> { /* T=Account, ID=Long */ }
```

> **NOTE**
> Type arguments must be **reference types**. `List<int>` is illegal; use `List<Integer>` (with autoboxing).

Generic methods

A generic method declares **its own** type parameter before the return type. It can live in any class, generic or not, and is often `static`.

```
public class Utils {
    public static <T> void printAll(T[] items) {                // <T> before return type
        for (T item : items) System.out.print(item + " ");
        System.out.println();
    }

    public static <T> void swap(T[] arr, int i, int j) {
        T tmp = arr[i]; arr[i] = arr[j]; arr[j] = tmp;
    }

    public static <K, V> Map<V, K> invert(Map<K, V> map) {
        Map<V, K> out = new HashMap<>();
        map.forEach((k, v) -> out.put(v, k));
        return out;
    }
}

Utils.printAll(new Integer[]{1, 2, 3});       // T inferred as Integer
Utils.printAll(new String[]{"a", "b"});       // T inferred as String
Utils.<String>printAll(new String[]{"x"});    // explicit type argument (rarely needed)
```

Generic class	Generic method
Type parameter declared on the class: `class Box<T>`	Declared on the method: `<T> void m(T t)`
Fixed when the object is created: `new Box<String>()`	Inferred for **each call** from the arguments
Shared by all instance members	Scoped to that one method; can be `static`

Bounded type parameters

`<T extends X>` restricts `T` to `X` or its subtypes, so you can call `X`'s methods on `T`. (`extends` is used for both classes and interfaces here.)

```
class Stats<T extends Number> {                        // only numeric types
    private final List<T> values = new ArrayList<>();
    void add(T v) { values.add(v); }
    double sum() {
        double s = 0;
        for (T v : values) s += v.doubleValue();       // Number's method is available
        return s;
    }
}
Stats<Integer> si = new Stats<>();                     // OK
// Stats<String> ss = new Stats<>();                   // COMPILE ERROR: String isn't a Number

static <T extends Comparable<T>> T max(List<T> list) {    // bounded generic method
    T best = list.get(0);
    for (T t : list) if (t.compareTo(best) > 0) best = t;
    return best;
}
max(List.of(3, 9, 4));            // 9
max(List.of("pear", "apple"));    // pear

class Ledger<T extends Number & Comparable<T>> { }  // class bound first, then interfaces
```

Rules for multiple bounds: at most **one class**, which must come **first**, followed by any number of interfaces, separated by `&`.

Generics are invariant

```
List<Integer> ints = new ArrayList<>();
// List<Number> nums = ints;          // COMPILE ERROR even though Integer IS-A Number
// Why: otherwise nums.add(3.14) would put a Double into a List<Integer>.

Object[] objs = new String[2];        // arrays ARE covariant (compiles)...
objs[0] = 42;                         // ...but throws ArrayStoreException at runtime
```

`List<Integer>` is **not** a subtype of `List<Number>`. To write methods that accept "a list of some kind of Number", use **wildcards**.

Wildcards: ?, ? extends, ? super

Wildcard	Accepts	Can read as	Can add
`List<?>` (unbounded)	A List of anything	`Object`	Nothing (except `null`)
`List<? extends Number>` (upper bound)	`List<Number>`, `List<Integer>`, `List<Double>`...	`Number`	Nothing (except `null`)
`List<? super Integer>` (lower bound)	`List<Integer>`, `List<Number>`, `List<Object>`	Only `Object`	`Integer` (and its subtypes)

```
static void printAll(List<?> list) {                 // read-only, any type
    for (Object o : list) System.out.print(o + " ");
}

static double total(List<? extends Number> nums) {   // PRODUCER: we read Numbers out
    double sum = 0;
    for (Number n : nums) sum += n.doubleValue();
    // nums.add(1);                                  // COMPILE ERROR: exact type unknown
    return sum;
}
total(List.of(1, 2, 3));          // List<Integer> OK
total(List.of(1.5, 2.5));         // List<Double>  OK

static void fillIds(List<? super Integer> sink) {    // CONSUMER: we put Integers in
    for (int i = 1; i <= 3; i++) sink.add(i);
    Object o = sink.get(0);                          // reading gives only Object
}
fillIds(new ArrayList<Integer>());
fillIds(new ArrayList<Number>());
fillIds(new ArrayList<Object>());
```

PECS: Producer Extends, Consumer Super

> **REMEMBER**
>
> **PECS**:
>
> - If a parameter **produces** values you **read** → `? extends T`.
> - If a parameter **consumes** values you **write** → `? super T`.
> - If it does both → use the exact type `T`, with no wildcard.

```
// Real JDK signature that follows PECS:
public static <T> void copy(List<? super T> dest, List<? extends T> src)

List<Number> dest = new ArrayList<>(List.of(0, 0, 0));
List<Integer> src = List.of(7, 8, 9);
Collections.copy(dest, src);             // dest consumes, src produces
```

10.8 Type Erasure

Generics exist **only at compile time**. The compiler checks the types, inserts casts, and then **erases** type parameters: `T` becomes `Object` (or its first bound, such as `Number`), and `List<String>` becomes plain `List` in the bytecode. This kept Java 5 generics backward-compatible with old code.

```
List<String> a = new ArrayList<>();
List<Integer> b = new ArrayList<>();
System.out.println(a.getClass() == b.getClass());   // true: both are just ArrayList at runtime
```

Consequences of erasure (things you can't do)

Not allowed	Why	Workaround
`new T()`	The type is unknown at runtime	Pass a `Supplier<T>` or a `Class<T>`
`new T[10]` / `new List<String>[10]`	Arrays need the runtime element type	`(T[]) new Object[n]` with care, or use a `List<T>`
`obj instanceof List<String>`	No runtime type argument	`instanceof List<?>`
`static T field;`	The static is shared by all parameterisations	Make it non-static, or a generic method
`List<int>`	Type arguments must be objects	`List<Integer>`, or primitive streams/arrays
Overloads `m(List<String>)` and `m(List<Integer>)`	Both erase to `m(List)`: name clash	Use different method names
`catch (T e)` / generic exception classes	Catch needs the runtime type	Not possible

```
static <T> T create(Supplier<T> factory) { return factory.get(); }
StringBuilder sb = create(StringBuilder::new);            // workaround for new T()

static <T> T[] newArray(Class<T> type, int n) {
    @SuppressWarnings("unchecked")
    T[] arr = (T[]) java.lang.reflect.Array.newInstance(type, n);
    return arr;
}
```

PITFALL

Raw types (`List list = new ArrayList();`) switch off generic checks and produce "unchecked" warnings. Mixing raw and generic code lets wrong types slip in until a `ClassCastException` appears far away (*heap pollution*). Don't use raw types in new code.

10.9 Predict the Output

```
class Outer { int v = 1; class Inner { int v = 2; int get() { return Outer.this.v + v; } } }
System.out.println(new Outer().new Inner().get());          // 3

enum Size { S, M, L }
System.out.println(Size.M.ordinal() + " " + Size.valueOf("L").compareTo(Size.S));   // 1 2
System.out.println(Size.values().length);                   // 3
try { Size.valueOf("XL"); } catch (IllegalArgumentException e) { System.out.println("IAE"); }

List<String> s = new ArrayList<>(); List<Integer> i = new ArrayList<>();
System.out.println(s.getClass() == i.getClass());           // true (erasure)

Runnable r = new Runnable() {
    public void run() { System.out.println(getClass().getSimpleName().isEmpty()); }
};
r.run();                                    // true (anonymous class: empty simple name)

List<? extends Number> nums = List.of(1, 2.5);
Number first = nums.get(0);
System.out.println(first);                                  // 1
```

10.10 Interview Questions

Q. What are the types of nested classes in Java?

Static nested classes, and three kinds of inner (non-static) classes: member, local and anonymous. Inner classes hold a reference to an enclosing instance; static nested classes don't.

Q. What is the difference between a static nested class and an inner class?

A static nested class doesn't need an outer instance and can only access the outer class's static members directly. A member inner class is tied to an outer instance (`outer.new Inner()`) and can access all its members via `Outer.this`.

Q. Why must local variables used in anonymous classes or lambdas be effectively final?

The inner code captures a **copy** of the value. Allowing later changes would make the copy and the original diverge, so Java requires the variable never to be reassigned.

Q. How is an anonymous class different from a lambda?

Lambdas only implement functional interfaces, don't create a separate class file, have no state of their own, and `this` refers to the enclosing instance. Anonymous classes can implement any interface or extend any class, hold fields, and define several methods; their `this` is the anonymous object.

Q. Can an enum extend a class or implement an interface? Can it have a constructor?

It can't extend a class (it already extends `java.lang.Enum`), but it can implement interfaces. It can have fields, methods and constructors, which are implicitly private and invoked only for its constants.

Q. What is the difference between `name()`, `toString()` and `ordinal()`?

`name()` returns the declared identifier (final). `toString()` returns the same by default but can be overridden for display. `ordinal()` returns the zero-based declaration position, which shouldn't be persisted.

Q. Why is an enum the best way to implement a singleton?

The JVM guarantees one instance per constant. It is thread-safe at class initialisation, protected against reflection attacks, and handles serialization correctly without extra code.

Q. What are generics and why use them?

Parameterised types that provide compile-time type checking, remove casts, and let one class or method work with many types safely.

Q. What is type erasure?

The compiler removes generic type information after checking it, replacing type parameters with their bounds (or `Object`) and inserting casts. At runtime, `List<String>` and `List<Integer>` are both just `List`.

Q. Why is `List<Integer>` not a subtype of `List<Number>`?

Generics are invariant. If it were a subtype, you could add a `Double` through the `List<Number>` reference into a list that must only contain Integers.

Q. Explain `? extends T` vs `? super T` (PECS).

`? extends T` accepts T or its subtypes; you can safely read `T`s but not add. `? super T` accepts T or its supertypes; you can add `T`s but only read `Object`s. Producer → extends, consumer → super.

Q. Why can't we create `new T()` or `new T[]`?

The actual type of `T` is erased and unknown at runtime. Pass a `Supplier<T>` or a `Class<T>` (reflection) instead.

10.11 Quick Revision

- Nested classes: **static nested** (no outer instance, `new Outer.Nested()`), **member inner** (`outer.new Inner()`, `Outer.this`), **local** (inside a method), **anonymous** (`new X() { }`).
- Inner classes hold a reference to the outer object, so prefer static nested when possible (e.g. the Builder pattern).
- Captured locals must be **effectively final** (for local and anonymous classes and lambdas).
- Lambda for functional interfaces; anonymous class when you need state, several methods, or to extend a class.
- Enum: fixed constants, extends `Enum`, can implement interfaces, private constructors, one instance per constant, compare with `==`.
- `values()`, `valueOf()` (case-sensitive; IAE if unknown), `name()`, `ordinal()` (never persist it).
- Enums can carry fields, constructors, methods and constant-specific bodies. `EnumSet`/`EnumMap` are fast enum collections. An enum is the best singleton.
- Generics: compile-time safety, no casts. `<>` diamond; reference types only; avoid raw types.
- Generic method: `<T>` before the return type, inferred per call.
- Bounds: `<T extends Number & Comparable<T>>` (one class first, then interfaces).
- Invariance: `List<Integer>` is not a `List<Number>`. Wildcards: `?`, `? extends` (read), `? super` (write). **PECS**.
- Erasure: no `new T()`, no generic arrays, no `instanceof List<String>`, no primitive type arguments, overload clashes.